Verilog Source Code

Design and Simulation of a 4-Bit Counter

Minor Project 03 | Thipperudra K T | 4SM23EE037 | 24 October 2026

Project Components:

1. HDL code (Verilog) for the 4-bit counter

File: `src/counter_4bit.v` (29 lines) - synchronous up/down counter with synchronous reset, enable and direction input.

```
1  `timescale 1ns / 1ps
2  //------------------------------------------------------------
3  // Module : counter_4bit
4  // Brief  : 4-bit synchronous up/down counter
5  //          - synchronous active-high reset
6  //          - count enable
7  //          - up_down = 1 : count up, up_down = 0 : count down
8  //------------------------------------------------------------
9  module counter_4bit (
10     input  wire       clk,       // clock, rising-edge triggered
11     input  wire       rst,       // synchronous reset (active high)
12     input  wire       en,        // count enable
13     input  wire       up_down,   // 1 = up, 0 = down
14     output reg  [3:0] count      // 4-bit count output
15 );
16
17     always @(posedge clk) begin
18         if (rst)
19             count <= 4'b0000;              // reset has highest priority
20         else if (en) begin
21             if (up_down)
22                 count <= count + 4'd1;     // 15 wraps to 0
23             else
24                 count <= count - 4'd1;     // 0 wraps to 15
25         end
26         // en = 0 : count holds its value
27     end
28
29 endmodule
```

2. Testbench for functional simulation

File: `tb/tb_counter_4bit.v` (41 lines) - 10 ns clock, 48 cycles (480 ns), seven stimulus phases, $monitor console output and VCD dump.

```
1  `timescale 1ns / 1ps
2  module tb_counter_4bit;
3
4      reg        clk, rst, en, up_down;
5      wire [3:0] count;
6
7      // Device under test
8      counter_4bit dut (.clk(clk), .rst(rst), .en(en),
9                        .up_down(up_down), .count(count));
10
11     // 10 ns clock (100 MHz)
12     initial clk = 1'b0;
13     always  #5 clk = ~clk;
```


© 2026 UnloxAcademy. All Rights Reserved.

Made by recycled paper, because we care.

```
14
15      // Console monitor
16      initial $monitor("t=%0d ns | rst=%b en=%b up_down=%b | count=%2d (%b)",
17                       $time, rst, en, up_down, count, count);
18
19      // Stimulus - inputs change on the falling edge
20      initial begin
21          $dumpfile("counter_4bit.vcd");
22          $dumpvars(0, tb_counter_4bit);
23
24          rst = 1; en = 0; up_down = 1;      // 1. reset
25          repeat (2)  @(negedge clk);
26          rst = 0; en = 1;                   // 2. count up (wraps 15 -> 0)
27          repeat (17) @(negedge clk);
28          en = 0;                            // 3. hold
29          repeat (3)  @(negedge clk);
30          en = 1; up_down = 0;               // 4. count down (wraps 0 -> 15)
31          repeat (18) @(negedge clk);
32          up_down = 1;                       // 5. count up again
33          repeat (5)  @(negedge clk);
34          rst = 1;                           // 6. reset in the middle of counting
35          @(negedge clk);
36          rst = 0;                           // 7. resume counting
37          repeat (2)  @(negedge clk);
38          #3 $finish;
39      end
40
41  endmodule
```

3. How to run

```
iverilog -o sim/sim src/counter_4bit.v tb/tb_counter_4bit.v
cd sim && vvp sim && cd ..
gtkwave sim/counter_4bit.vcd
```

GitHub repository: _____

© 2026 UnloxAcademy. All Rights Reserved.

Made by recycled paper, because we care.